

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«БАЛТИЙСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ ИМЕНИ ИММАНУИЛА КАНТА»

ОТЧЁТ

по учебно-технологической практике

**Тема: «Разработка сетевого модуля игровой платформы Bobux с авторизацией,
каталогом пользовательского контента и синхронизацией игровых данных»**

Студент: [ФИО студента]

Группа: [номер группы]

Руководитель практики: [ФИО руководителя]

Калининград 2026

Содержание

Введение

1. Описание постановки задачи
2. Структура ролей пользователей
3. Обоснование выбора технологий
4. Общая архитектура сетевого модуля
5. База данных и основные коллекции
6. Реализация авторизации и профилей
7. Каталог пользовательского контента
8. Работа с файлами и изображениями
9. Социальный модуль
10. Регистрация активных игровых серверов
11. Сетевое взаимодействие клиента и сервера
12. Серверная валидация и безопасность
13. Административная статистика
14. Тестирование
15. Результаты работы

Заключение

Список использованных источников

Приложения

Введение

Целью учебно-технологической практики является получение практического опыта разработки программных модулей, работающих с базой данных, пользовательскими аккаунтами, файлами и сетевыми запросами. В качестве предметной области выбрана игровая платформа Bobux. Платформа включает клиент на Godot, серверный API, базу данных PocketBase, файловое хранилище, выделенный игровой сервер и лаунчер обновлений.

В рамках практики рассматривается не вся игровая платформа целиком, а отдельный законченный программный модуль: сетевой модуль игровой платформы. Он отвечает за регистрацию и авторизацию пользователей, хранение профилей, каталог пользовательских игр и предметов, синхронизацию аватара, работу с друзьями, регистрацию активных игровых серверов и загрузку файлов.

При разработке применялись нейросетевые агенты как вспомогательный инструмент для анализа кода, поиска ошибок и ускорения реализации. Постановка задачи, выбор архитектуры, интеграция, проверка и итоговое оформление выполнялись автором проекта.

1. Описание постановки задачи

Необходимо разработать сетевой модуль для игровой платформы, который обеспечивает взаимодействие игрового клиента с серверной инфраструктурой. Модуль должен выполнять регистрацию и авторизацию пользователей, хранить профили и данные аватара, обеспечивать CRUD-операции для пользовательского контента, хранить карты, модели и предметы аватара, поддерживать список друзей, регистрировать активные игровые серверы и предоставлять административную статистику.

Поставленная задача соответствует общим требованиям учебно-технологической практики: используются база данных, авторизация, управление записями, загрузка файлов, поиск и серверная проверка данных.

2. Структура ролей пользователей

Роль	Возможности
Неавторизованный пользователь	Открытие клиента, переход к регистрации или авторизации
Авторизованный игрок	Лобби, запуск игр, профиль, аватар, инвентарь, друзья
Создатель контента	Публикация карт, моделей и предметов аватара
Администратор	Доступ к защищённой статистике и состоянию сервера

3. Обоснование выбора технологий

Технология	Назначение
Godot 4.7 / GDScript	Клиент игровой платформы, UI, лобби и подключение

	к игровому серверу
Node.js / Express	Серверный HTTP API, валидация, маршрутизация запросов
PocketBase	База данных и коллекции пользовательских данных
WebSocket	Сетевое взаимодействие в игровой сессии
Nginx	Проксирование HTTP и WebSocket-запросов
PM2	Запуск и восстановление Node.js API на VPS

4. Общая архитектура сетевого модуля

Сетевой модуль построен по многоуровневой архитектуре. Клиент Godot не обращается к базе данных напрямую. Все операции выполняются через серверный API. Это повышает безопасность и позволяет централизованно контролировать валидацию, права доступа, формат данных и загрузку файлов.

Основные уровни архитектуры: клиентский уровень Godot, серверный API Node.js, база данных PocketBase, файловое хранилище, игровой WebSocket-сервер и инфраструктурный слой VPS.

Архитектура сетевого модуля игровой платформы Bobux

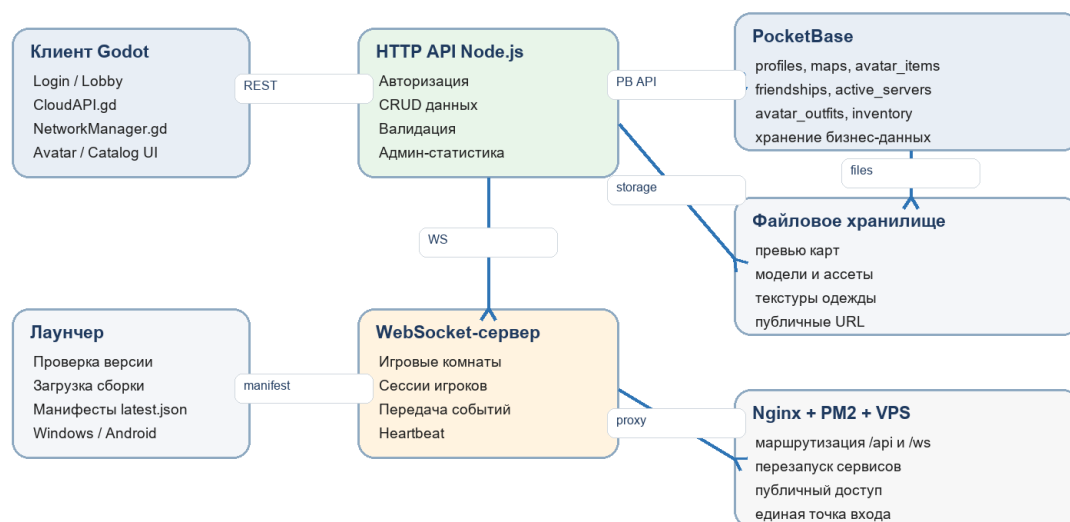


Рисунок 1 — Архитектура сетевого модуля игровой платформы Bobux

5. База данных и основные коллекции

Коллекция	Назначение
profiles	Профили игроков, имя, статус, текущая игра, данные аватара

maps	Пользовательские карты и игровые режимы
active_servers	Активные игровые комнаты и heartbeat-состояние серверов
model_assets	Загруженные пользовательские модели
avatar_items	Одежда, аксессуары и предметы аватара
avatar_outfits	Текущее состояние внешнего вида пользователя
user_inventory	Связь пользователя с полученными предметами
friendships	Подтверждённые дружеские связи
friend_requests	Входящие и исходящие заявки в друзья
follows	Подписки между пользователями
asset_likes	Лайки карт, моделей и предметов

6. Реализация авторизации и профилей

Авторизация реализована через совместимый API-слой. Клиент отправляет логин и пароль на серверный маршрут `/api/auth/v1/token`. Сервер проверяет пользователя через PocketBase и возвращает access token. Клиент сохраняет сессию локально и использует bearer token для последующих запросов.

После входа выполняется загрузка профиля пользователя. В профиле хранится имя, статус, текущая игра и базовые данные аватара. Отдельно хранится `avatar_outfits`, где содержится актуальная одежда, цвета тела и список надетых предметов. Такое разделение позволяет не перегружать основной профиль тяжёлыми данными и быстрее открывать лобби.

- загрузка outfit после входа выполняется фоном и не блокирует логин;
- публичные списки друзей получают облегчённые профили без тяжёлых inline-данных;
- сервер обрезает слишком крупные thumbnails в ответах списка карт и профилей;
- общий HTTP timeout уменьшен, чтобы зависший запрос не держал интерфейс слишком долго.

7. Каталог пользовательского контента

Модуль поддерживает карты и игровые режимы, модели, предметы аватара, изображения и превью, элементы инвентаря. Публикация контента проходит через серверный API. Клиент отправляет данные, сервер проверяет пользователя, сохраняет запись в нужной коллекции и при необходимости сохраняет файл в публичное хранилище.

8. Работа с файлами и изображениями

Файлы не хранятся напрямую внутри базы данных в виде больших бинарных объектов. Для них используется файловое хранилище, а в базе сохраняются пути или публичные URL. Такой подход снижает нагрузку на базу данных и ускоряет выдачу списков. Сервер дополнительно контролирует

размер inline-данных, чтобы старые или слишком крупные изображения не замораживали клиентский интерфейс.

9. Социальный модуль

Социальная часть сетевого модуля включает отправку заявки в друзья, получение входящих и исходящих заявок, принятие и отклонение заявки, удаление друга, список друзей и подсчёт социальных связей. Социальные RPC-маршруты работают через серверный access token. Клиент не получает доступа к базе напрямую.

10. Регистрация активных игровых серверов

Для отображения активных игровых режимов используется коллекция `active_servers`. Игровой сервер или клиентский хост отправляет heartbeat-записи, где указывается карта, комната, количество игроков, хост и время последней активности. Лобби запрашивает список активных серверов и может показывать, где прямо сейчас есть игроки.

11. Сетевое взаимодействие клиента и сервера

В клиенте Godot сетевой модуль разделён на две основные части: `CloudAPI.gd` выполняет HTTP-операции, а `NetworkManager` отвечает за WebSocket-подключение к игровой сессии. HTTP используется для долговременных данных, WebSocket — для событий игрового процесса в реальном времени.

1. Пользователь вводит логин и пароль.
2. Клиент отправляет запрос на `/api/auth/v1/token`.
3. Сервер проверяет данные через PocketBase.
4. Клиент получает токен и загружает профиль.
5. Лобби открывается без ожидания тяжёлых данных.
6. Одежда и дополнительные данные догружаются в фоне.
7. При запуске режима клиент подключается к WebSocket-серверу.

12. Серверная валидация и безопасность

- клиент не имеет прямого доступа к PocketBase;
- все операции проходят через Express API;
- административная статистика защищена `superuser`-токеном;
- `service credentials` не передаются клиенту;
- загрузка файлов проходит через серверные маршруты;
- пути файлов нормализуются и сохраняются в безопасной директории;
- тяжёлые inline-данные очищаются перед выдачей публичным спискам;
- сервер проверяет текущего пользователя по `bearer token`.

13. Административная статистика и фактическое состояние системы

Для проверки состояния сетевого модуля реализован защищённый endpoint `/api/admin/stats`. Он собирает статистику по аккаунтам, профилям, картам, предметам, инвентарю, дружбам, активным серверам и размеру хранилищ.

Показатель	Значение
Зарегистрированные auth-пользователи	612
Профили игроков	654
Именованные профили	607
Пользовательские карты	86
Создатели карт	42
Суммарные посещения карт	680
Суммарные лайки карт	57
Загруженные модели	8
Предметы аватара	83
Записи инвентаря	1077
Дружеские связи	172
Размер данных PocketBase	2005.53 MB
Размер публичного файлового хранилища	411.25 MB

14. Тестирование

- синтаксическая проверка Node.js API командой `node --check server.js`;
- проверка проекта Godot через `Godot --headless --check-only`;
- проверка доступности `/api/health`;
- проверка защищённой статистики `/api/admin/stats`;
- проверка загрузки списков профилей и карт без тяжёлых inline-данных;
- ручная проверка входа, лобби, каталога, друзей и аватара.

15. Результаты работы

- реализованы авторизация и управление сессиями;
- организовано хранение пользовательских профилей;
- создан каталог карт, моделей и предметов;
- реализована загрузка и выдача файлов;
- добавлена синхронизация аватара и инвентаря;
- реализована социальная система друзей;

- добавлена регистрация активных игровых серверов;
- добавлена защищённая административная статистика;
- оптимизированы тяжёлые сетевые ответы.

Заключение

Разработанный сетевой модуль решает основные задачи серверной части игровой платформы: хранение данных, авторизация, синхронизация клиента и сервера, публикация пользовательского контента и администрирование. Работа позволила применить навыки проектирования базы данных, разработки API, работы с сетевыми протоколами, файловым хранилищем и клиент-серверной архитектурой.

В дальнейшем модуль можно развивать: добавить полноценную веб-админку, расширить систему ролей, добавить модерацию пользовательского контента, улучшить мониторинг активных игровых серверов и внедрить автоматическое резервное копирование.

Список использованных источников

8. Методические материалы по учебно-технологической практике, предоставленные руководителем практики.
9. Документация Godot Engine: <https://docs.godotengine.org/>
10. Документация Node.js: <https://nodejs.org/docs/>
11. Документация Express: <https://expressjs.com/>
12. Документация PocketBase: <https://pocketbase.io/docs/>
13. MDN Web Docs: материалы по HTTP, REST и WebSocket.

Приложение А. Основные файлы проекта

Файл	Назначение
autoload/cloud_api.gd	HTTP-клиент Godot для работы с серверным API
autoload/client_network.gd	WebSocket-клиент и подключение к игровой сессии
autoload/user_session.gd	Локальная пользовательская сессия
autoload/game_state.gd	Состояние аватара и выбранной карты
services/bobux_api/server.js	Основной Node.js/Express API
server/server_main.gd	Выделенный игровой WebSocket-сервер
scripts/lobby/lobby.gd	Лобби, каталог, профиль и социальные разделы
tools/deploy_bobux_release.ps1	Скрипт подготовки релиза
ops/vps/bobux_nginx.conf	Конфигурация Nginx для API и WebSocket